Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Informatyka stosowana, studia II stopnia

Mobilne systemy informatyczne – Projekt

Raport ze Sprintu 2: podstawowy interfejs, nawigacja, model danych

WroTransit – system monitorowania komunikacji miejskiej we Wrocławiu w czasie rzeczywistym

Autorzy:

Andrzej Manderla

Nr albumu: 294 692

Igor Wojtun

Nr albumu: 272 652

Prowadzący:

dr inż. Zbigniew Telec

Wrocław, 9 listopada 2026

Spis treści

1 Zakres sprintu **2**

2 Model danych **2**

3 Import danych **4**

4 API **5**

5 Aplikacja **6**

6 Testy i CI **7**

7 Ograniczenia **8**

8 Plan na Sprint 3 **8**

A Wykorzystanie narzędzi sztucznej inteligencji **9**

1. Zakres sprintu

Sprint 2 (tygodnie 5–6) obejmował podstawowy interfejs, nawigację i model danych. Oprócz tego zrealizowaliśmy część zakresu Sprintu 3: API z wyszukiwaniem i tablicą odjazdów, dzięki czemu aplikacja działa już na prawdziwych danych rozkładowych, a nie na danych testowych.

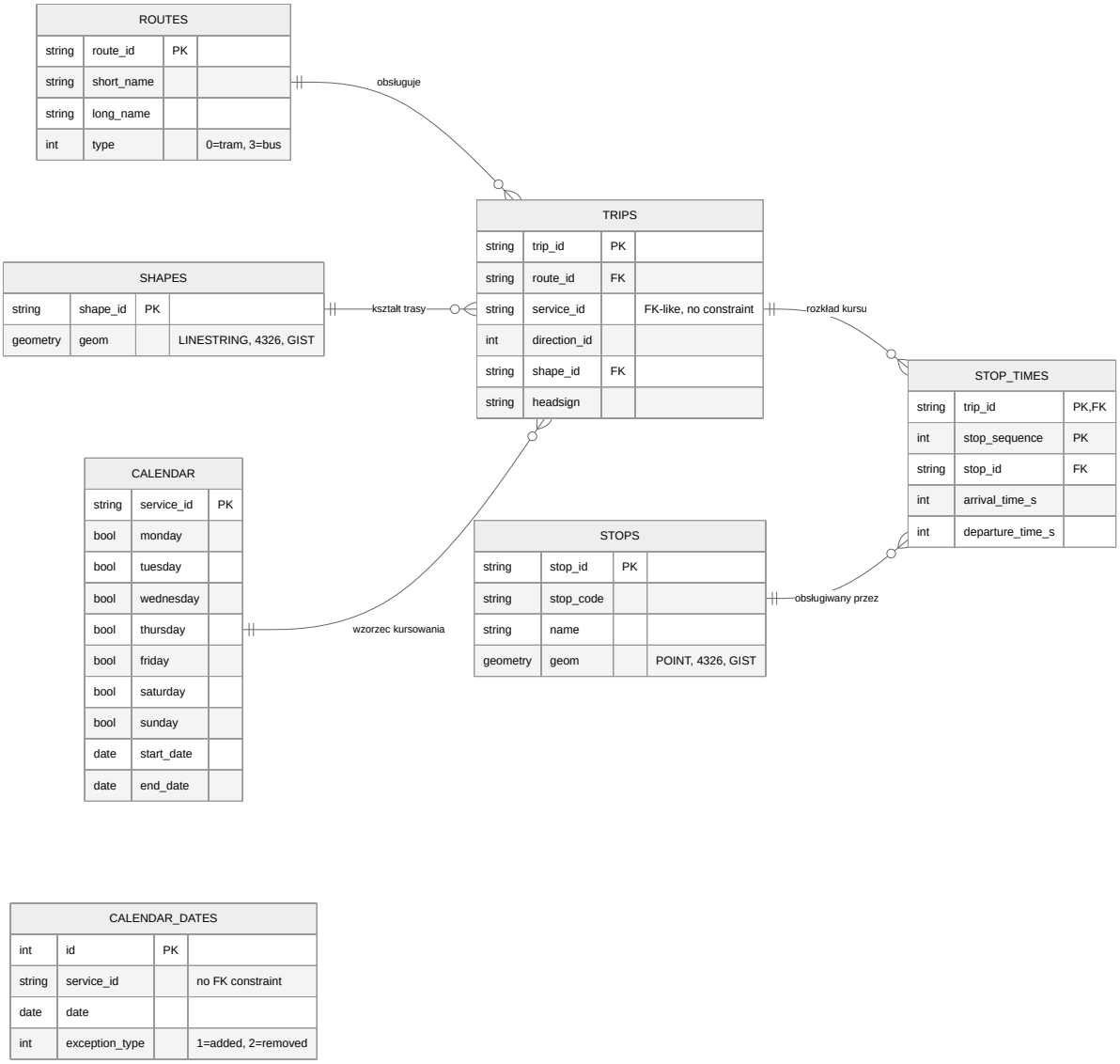
Tabela 1: Zrealizowane prace

Obszar	**Wynik**
Model danych	Pełny schemat GTFS z geometrią PostGIS, schemat pod pozycje pojazdów i konta (rozdz. 2)
Import danych	Import rozkładu GTFS Wrocławia do bazy (rozdz. 3)
API	Przystanki, linie, kształty tras, odjazdy (rozdz. 4)
Aplikacja	Mapa, wyszukiwanie, odjazdy, przebieg linii, ulubione, nawigacja (rozdz. 5)
Testy i CI	11 testów na PostgreSQL+PostGIS, CI z bazą (rozdz. 6)

2. Model danych

W Sprincie 0 zakładaliśmy uproszczony model rozkładu. Ostatecznie odwzorowaliśmy pełny schemat GTFS: `stops`, `routes`, `trips`, `stop_times`, `shapes`, `calendar`, `calendar_dates`. Bez kalendarza kursowania nie da się poprawnie ustalić, które kursy jadą danego dnia, a bez kształtów tras – wyznaczać opóźnień w Sprincie 4.

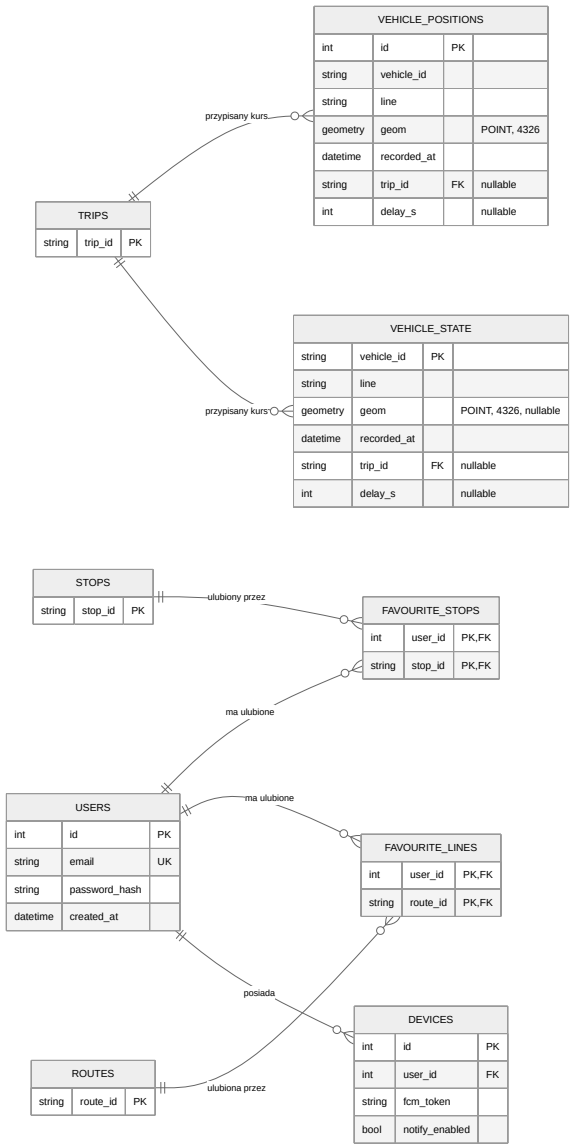
Geometrie są przechowywane jako typy PostGIS w układzie WGS 84 (EPSG:4326): przystanki jako `POINT`, kształty tras jako `LINESTRING`. Na obu kolumnach są indeksy GIST pod zapytania przestrzenne (przystanki w widocznym obszarze mapy, przystanki w pobliżu).



Rysunek 1: ERD – tabele rozkładowe (GTFS)

Opracowano z użyciem NSI (Claude (Anthropic)): diagram ERD wygenerowany na podstawie schematu bazy.

Tabele na pozycje pojazdów (`vehicle_positions`, `vehicle_state`) i konta użytkowników (`users`, `favourite_stops`, `favourite_lines`, `devices`) mają już schemat, ale pozostają puste do Sprintu 3 (pobieranie pozycji) i Sprintu 5 (autoryzacja).



Rysunek 2: ERD – pozycje pojazdów i konta użytkowników (tabele rozkładowe tylko z kluczem)

Opracowano z użyciem NSI (Claude (Anthropic)): diagram ERD wygenerowany na podstawie schematu bazy.

Decyzje projektowe:

- kolumny `service_id` w `trips` i `calendar_dates` nie mają klucza obcego – GTFS dopuszcza serwis zdefiniowany wyłącznie wyjątkami w `calendar_dates.txt`, bez wpisu w `calendar.txt`; sztywny FK blokowałby import poprawnych plików;
- czasy w `stop_times` są zapisane jako sekundy od północy, bo GTFS dopuszcza godziny powyżej 24:00 dla kursów nocnych.

3. Import danych

Rozkład pochodzi z kopii feedu GTFS Wrocławia w Mobility Database, odświeżanej codziennie. Import czyta archiwum ZIP i w całości odtwarza tabele rozkładowe przy każdym urucho-

mieniu (`TRUNCATE` + wstawienie danych).

Tabela 2: Skala zaimportowanych danych

Tabela	Liczba wierszy
`stops`	2 429
`routes`	133
`trips`	41 790
`stop_times`	1 131 610

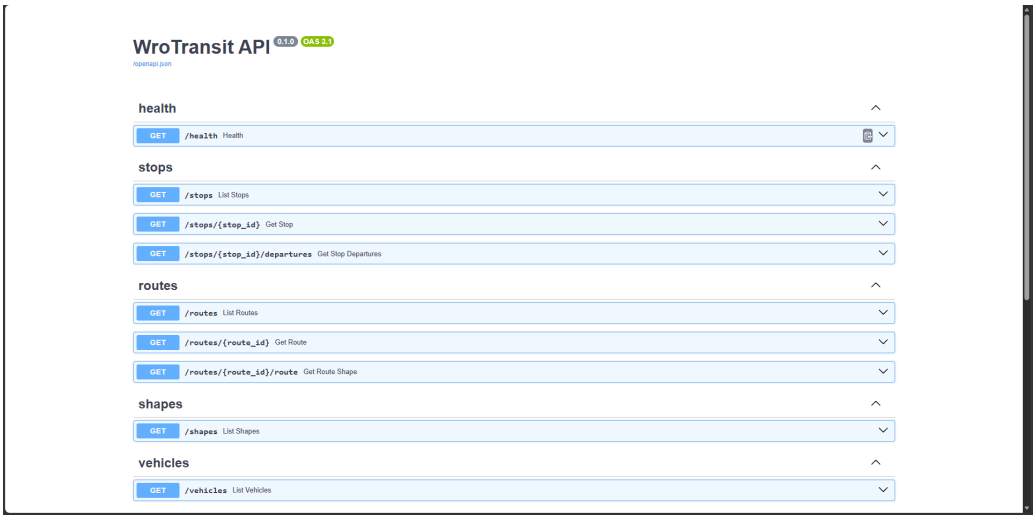
Przy ponad milionie wierszy `stop_times` wstawianie przez ORM było zbyt wolne, dlatego dane ładujemy poleceniem `COPY` (psycopg).

4. API

Tabela 3: Endpointy po Sprincie 2

Endpoint	Opis
`GET /health`	status usługi
`GET /stops?q=&bbox=`	wyszukiwanie po nazwie, przystanki w widocznym obszarze mapy
`GET /stops/{id}`	szczegóły przystanku
`GET /stops/{id}/departures`	najbliższe odjazdy wg rozkładu (dzień tygodnia i zakres dat kursowania)
`GET /routes?q=`	wyszukiwanie linii
`GET /routes/{id}`	szczegóły linii
`GET /routes/{id}/route`	uporządkowany przebieg przystanków linii
`GET /shapes`	geometrie tras z typem pojazdu (do rysowania na mapie)
`GET /vehicles`	szkielet – dane pozycji w Sprincie 3

CORS jest skonfigurowany pod wersję webową aplikacji. Dokumentacja OpenAPI generuje się automatycznie (rys. 3).

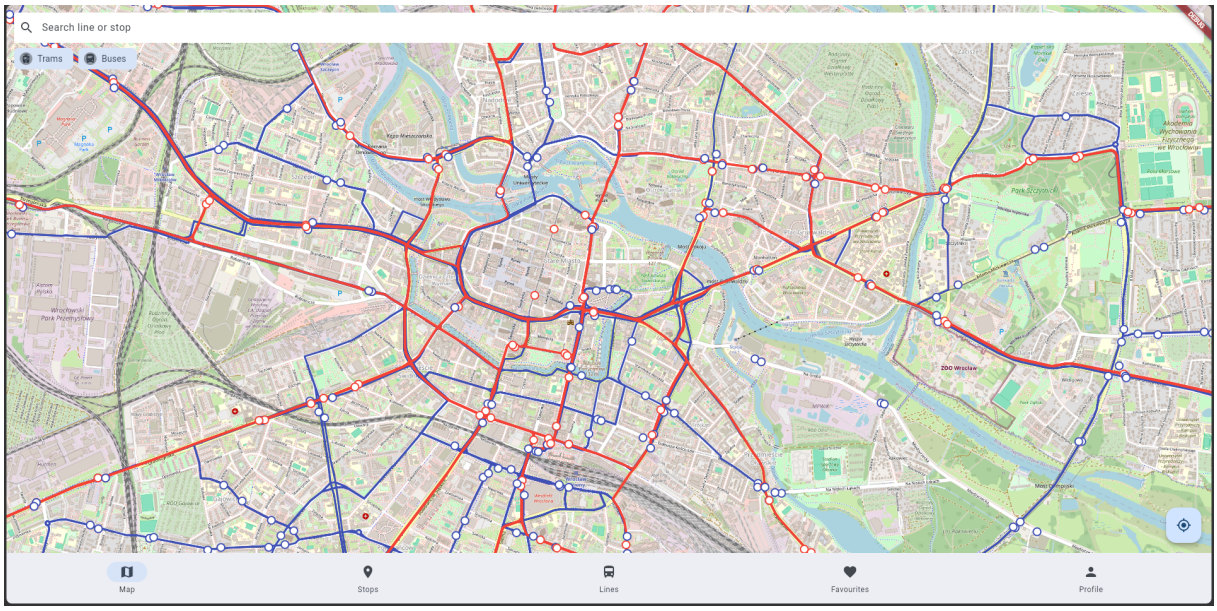


Rysunek 3: Dokumentacja API (Swagger UI)

5. Aplikacja

Aplikacja działa na trzech platformach: Android, web i Windows (desktop). Stan aplikacji jest zarządzany przez Riverpod. Zrealizowane ekrany:

- **Mapa** (rys. 4) – trasy linii (tramwaje na czerwono, autobusy na niebiesko) i przystanki w widocznym obszarze, filtr tramwaj/autobus, lokalizacja użytkownika z obsługą uprawnień;
- **Przystanki** i **Linie** (rys. 5) – wyszukiwanie z opóźnieniem zapytań (*debounce*), przejście do tablicy odjazdów i przebiegu linii;
- **Tablica odjazdów** – odjazdy z API (bez opóźnień, te w Sprincie 4);
- **Ulubione** (rys. 6) – na razie trzymane lokalnie w pamięci aplikacji, zapis na koncie w Sprincie 5.



Rysunek 4: Mapa: trasy tramwajowe i autobusowe, przystanki, filtr typu pojazdu

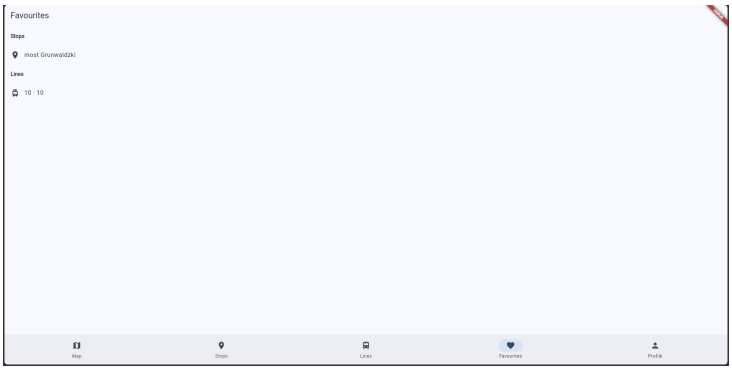


(a) Przystanki



(b) Linie

Rysunek 5: Listy z wyszukiwaniem

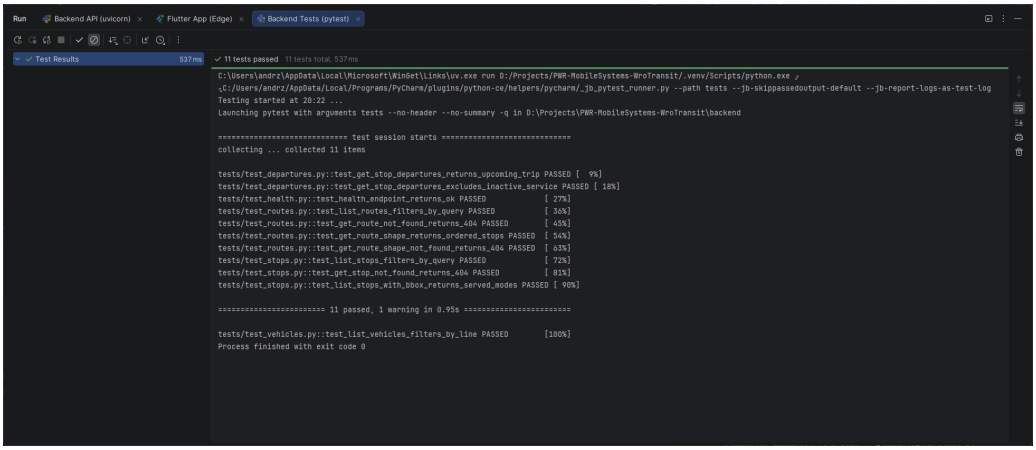


Rysunek 6: Ulubione przystanki i linie

6. Testy i CI

Testy backendu przenieśliśmy z SQLite na PostgreSQL z PostGIS, bo endpointy korzystają z funkcji przestrzennych, których SQLite nie ma. Zestaw obejmuje 11 testów: wyszukiwanie

i filtr `bbox` przystanków, odjazdy (w tym pomijanie nieaktywnego serwisu), linie i ich przebieg, obsługę błędu 404 oraz pojazdy. Potok CI w GitHub Actions uruchamia teraz kontener z bazą PostGIS.



Rysunek 7: Wynik testów backendu

7. Ograniczenia

- Zaimportowany snapshot GTFS ma kalendarz ważny do 12.12.2025, więc endpoint odjazdów zwraca pustą listę, dopóki nie zaimportujemy aktualnego archiwum. Logika działa poprawnie (potwierdzają to testy), nieaktualne są dane.
- Filtr tramwaj/autobus ukrywa trasy, a nie przystanki – jeden przystanek może obsługiwać oba typy pojazdów, więc zamiast go ukrywać oznaczamy typ kolorem obwódki.

8. Plan na Sprint 3

Pobieranie pozycji pojazdów i zapis do bazy z trybem odtwarzania nagranych danych, endpoint `/vehicles` na prawdziwych danych, pojazdy na mapie z odświeżaniem co 10 s, endpoint przystanków w pobliżu, import aktualnego archiwum GTFS.

A. Wykorzystanie narzędzi sztucznej inteligencji

Tabela 4: Wykorzystane narzędzia NSI

Narzędzie	Cel wykorzystania	Zakres / oznaczenie w raporcie
Claude (Anthropic)	Wygenerowanie diagramu ERD (Mermaid) na podstawie schematu bazy danych	Rysunki 1 i 2, oznaczone notą pod podpisem